

NMHS Hospital Management Information System

Deployment, Hosting &
System Administration Guide — Render.com

SYSTEM	NMHS-HMIS-V1.0
DOCUMENT	NMHS-HMIS-V1.0-DG
VERSION	2.0
DATE	2 October 2026
AUDIENCE	Technical administrators and developers
REPOSITORY INSPECTED	main + uncommitted Render/Cloudinary configuration

Source of truth. This guide describes the NMHS-HMIS-V1.0 repository as inspected on 2 October 2026: `render.yaml`, `backend/hmis/hmis/settings.py`, `hmis/environment.py`, `apps/core/storage.py` and `docs/DEPLOYMENT.md`. It replaces version 1.0, which described a self-hosted Nginx/systemd server. **Nothing in it has been deployed to Render, and the Cloudinary storage has not been run or tested.** No secrets appear in it; every credential is a placeholder.

Contents

Read this first	4
1. Architecture on Render	4
1.1 Services and exact commands (from render.yaml)	5
2. Requirements and Render plans	5
2.1 Plans and limits (Render's documentation, checked 2 October 2026)	5
3. Cloudinary for patient documents	6
3.1 How documents stay private	6
3.2 Temporary links and their expiry	6
3.3 Set up the Cloudinary account	7
4. Environment variables and secrets	7
4.1 Set by the Blueprint (group hmis-settings: api, worker, beat)	7
4.2 Set by Render itself	8
4.3 You enter them (prompted when the Blueprint is applied)	8
4.4 Optional — add on a service's Environment page	8
5. First deployment with the Render Blueprint	9
5.1 PostgreSQL and Key Value settings in the Blueprint	9
6. The Super Admin account	9
7. Custom domains, CORS, CSRF and VITE_API_BASE_URL	10
8. Celery worker and the single Celery Beat	10
9. Verifying documents upload and stay private	10
10. Health checks, logs and troubleshooting	11
10.1 Health check	11
10.2 Logs	11
10.3 Troubleshooting	11
11. Backups and recovery	12
11.1 Database	12
11.2 Documents	12
11.3 Disaster recovery	12
12. Updates, security and patient-data protection	12
12.1 Updating the HMIS	12
12.2 Security checklist	13
12.3 Patient-data protection	13
13. Smoke-test the HMIS after deploying	13
14. What has and has not been tested	14
Appendices	14

A. Local development (unchanged) 14

B. Useful commands (API Shell on Render, or locally) 14

C. Files that define the deployment 14

D. Departments seeded by migrations 15

E. Self-hosted alternative (not the primary method) 15

F. Support information 15

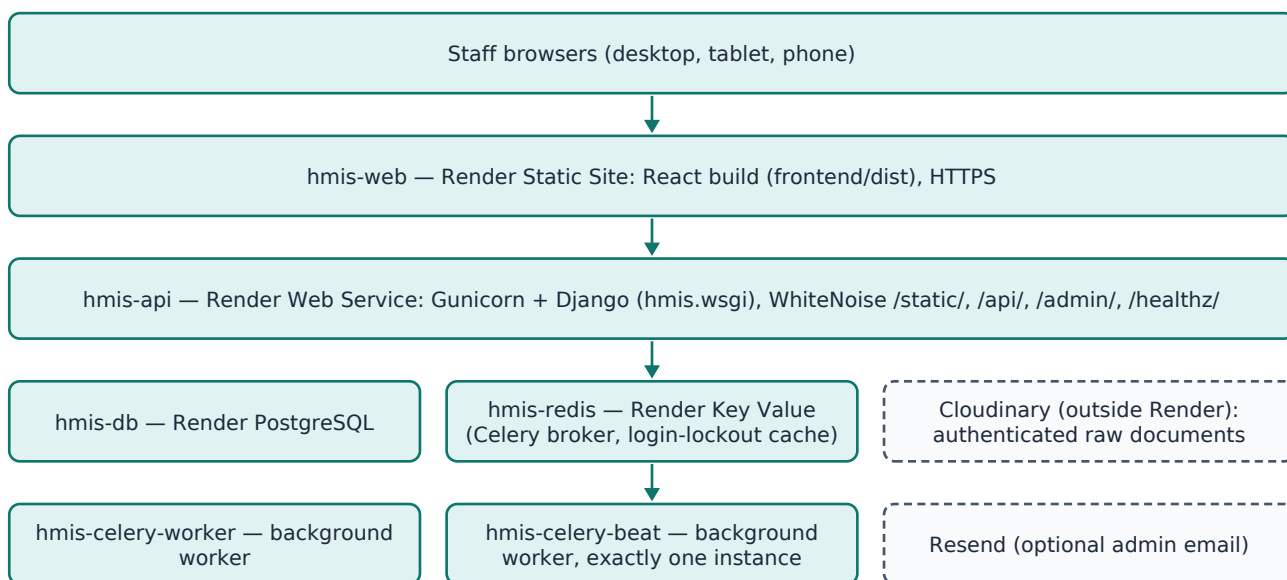
Read this first

IMPORTANT: Untested deployment

The repository is prepared for Render, but **no Render deployment has been made** and the **Cloudinary document storage has not been executed** — its tests were written and not run. Make the first deployment a rehearsal with test patients (chapter 13) before any real patient is registered.

Already done in the repository	You do (dashboard / accounts)
DJANGO_ENV selects development (SQLite) or production (PostgreSQL only — no SQLite fallback)	Create and secure the Cloudinary account (chapter 3)
Production refuses to start when a required variable is missing; the message names the variable, never its value	Apply the Blueprint and enter the secrets (chapters 4-5)
render.yaml Blueprint: API, static site, PostgreSQL, Key Value, Celery worker, one Celery Beat	Check Render plans and prices before applying
HTTPS behind Render's proxy, secure cookies, HSTS, WhiteNoise static files	Create the Super Admin (chapter 6)
Private Cloudinary storage with expiring links (apps/core/storage.py)	Custom domains, CORS, VITE_API_BASE_URL (chapter 7)
Trusted client-IP header for the login logout; optional Django Admin network allow-list	Optional: admin email, admin allow-list, log stream
manage.py check --deploy errors hmis.E001-E012 until production is secure	Run the smoke tests (chapter 13)

1. Architecture on Render



- **Two origins.** The React app and the API are separate services. The API authenticates with a token in the `Authorization` header — never a cookie — so CORS is all that connects them. Django Admin lives on the API origin, so the React route `/admin` and Django's `/admin/` do not collide.
- **Private network.** PostgreSQL and Key Value accept no internet connections (`ipAllowList: []`); the API and workers reach them over Render's private network in the same region.

- **No WebSockets.** The application has no `asgi.py`, no Channels and no WebSocket client; live data is polled (notification bell every 30 s, station queues every 60 s). WSGI under Gunicorn is the right server.
- **Documents** go to Cloudinary, not to Render's disk, which is wiped on every deploy.

1.1 Services and exact commands (from `render.yaml`)

Service	Build	Pre-deploy	Start
hmis-api (rootDir backend/hmis)	<code>pip install -r requirements.txt && python manage.py collectstatic --noinput</code>	<code>python manage.py migrate --noinput</code>	<code>gunicorn hmis.wsgi:application --bind 0.0.0.0:\$PORT --workers \${WEB_CONCURRENCY:-2} --timeout 60 --forwarded-allow-ips="*" --error-logfile -</code>
hmis-web (rootDir frontend)	<code>npm ci && npm run build, publish ./dist</code>	—	— (static; /* rewritten to /index.html)
hmis-celery-worker	<code>pip install -r requirements.txt</code>	—	<code>celery -A hmis worker -l info --concurrency 2</code>
hmis-celery-beat (numInstances: 1)	<code>pip install -r requirements.txt</code>	—	<code>celery -A hmis beat -l info --schedule /tmp/celerybeat-schedule</code>

- **Migrations run once per deploy**, in Render's pre-deploy step on a separate instance, before the new release takes traffic. A failure cancels the deploy and the previous release keeps serving. They are never in the start command, where every instance would migrate.
- `collectstatic` **is in the build**: the pre-deploy instance's files never reach the running service.
- Health check path: `/healthz/`.

2. Requirements and Render plans

	Development (unchanged)	Production on Render
Database	SQLite, backend/hmis/db.sqlite3	Render PostgreSQL 16 via DATABASE_URL
Server	<code>manage.py runserver</code> + <code>npm run dev</code> (Vite proxies <code>/api</code> , <code>/media</code>)	Gunicorn web service + static site
Documents	Local backend/hmis/media/	Cloudinary, private (chapter 3)
Static files	Served by <code>runserver</code>	WhiteNoise, compressed and cache-busted
Python / Node	Local venv; Node 18+	PYTHON_VERSION=3.12.8, NODE_VERSION=22
Setup needed	None	Everything in chapters 3–7

Python dependencies are pinned in `backend/hmis/requirements.txt` (Django 5.0.6, DRF 3.15.1, Celery 5.4.0, Gunicorn 22.0.0, `psycpg2-binary`, WhiteNoise 6.7.0, `cloudinary` 1.46.2). Development installs them too but never loads WhiteNoise or Cloudinary.

2.1 Plans and limits (Render's documentation, checked 2 October 2026)

Item	Requirement
Pre-deploy command (migrations)	Paid web service only
Background workers (Celery worker, Beat)	No free tier

Item	Requirement
Free PostgreSQL	Expires after 30 days , 1 GB, no backups — not for patient data
Free Key Value	Does not persist — a restart drops queued tasks
Free web service	Spins down after 15 min idle; no shell; no pre-deploy
Persistent disks	Paid, single instance — not used (Cloudinary instead)
Inbound IP rules for web services	Scale/Enterprise orgs only — hence <code>HMIS_ADMIN_ALLOWED_NETWORKS</code>
Log retention	Hobby 7 days, Pro 14, Scale/Enterprise 30 (log streams for longer)
PostgreSQL point-in-time recovery	Hobby workspace: past 3 days; Pro+: past 7 days

`render.yaml` uses the smallest paid sizes in Render's Blueprint reference (0.5c-512mb services, 0.5c-1g database, 256mb Key Value) in region `frankfurt`; every resource must share one region. Check current prices and capacity before applying. Cloudinary plans and limits are separate (chapter 3).

3. Cloudinary for patient documents

Uploaded results — Medical Tests & Diagnostics files, which unit reports are filed under (`medical_tests/YYYY/MM/`) — are patient documents. In production they are stored in Cloudinary by `apps/core/storage.py` (`PrivateCloudinaryStorage`), which is Django's default file storage, so every upload form, serializer, model and permission is unchanged.

3.1 How documents stay private

- Each file is uploaded **server-side** (signed with the API secret) as a *raw* asset with delivery type *authenticated*, under the folder `nmhs-hmis/media/` (`CLOUDINARY_FOLDER`). Cloudinary refuses an authenticated asset to anyone without a signature: its ordinary `res.cloudinary.com` address does not work. *raw* keeps images, PDFs, Word and Excel files byte-for-byte and allows no transformations.
- Each upload gets a unique name (a random suffix before the extension), so two uploads never overwrite each other (`overwrite=False`).
- The API **never returns a permanent address**. A document link is a Cloudinary **private download URL** — `https://api.cloudinary.com/v1_1/<cloud>/raw/download?...` — carrying `expires_at`. Cloudinary checks the expiry itself on every request, and these responses are not cached on its CDN.
- Signed CDN URLs are not used**: Cloudinary's signatures on those never expire. Time-limited CDN *tokens* require Cloudinary's Advanced plan or higher; the download URL used here is available on every plan.
- Links are created only by the serializers and views that already decide who may read the record, so **the existing permissions decide who gets a link**.
- A link contains the cloud name, the **API key** (an identifier) and a signature. The **API secret** never leaves the server: it is not in any link, any API response, any `VITE_*` variable, and the log-redaction filter removes it from log lines.

3.2 Temporary links and their expiry

Setting	Value
<code>CLOUDINARY_LINK_EXPIRY_SECONDS</code>	Default 900 (15 minutes); allowed 60–86400; set to 900 by the Blueprint DS
What expires	The link, not the document. After expiry Cloudinary refuses it; reopening the record in the HMIS issues a fresh link to whoever may read it.

Setting	Value
Shared links	A forwarded link works for anybody until it expires — keep the expiry short.

3.3 Set up the Cloudinary account

1. Create a Cloudinary account (or use the hospital's) with a **product environment for the HMIS only**. Check the plan's limits — in particular the maximum **raw** file size, which caps the size of a scan a desk can upload — and Cloudinary's data-protection terms and storage location for patient documents. **This is your decision.**
2. In the Cloudinary Console, open the API keys page and note the **cloud name**, an **API key** and its **API secret**. If the console lets you create additional keys, create one used by the HMIS alone so it can be revoked without affecting anything else.
3. Enable two-factor authentication for every console user and limit console access — a console user can see every document.
4. Review the **upload presets**. The HMIS uploads only through signed server-side calls and needs none; do not keep an *unsigned* preset, which would let anybody upload to the account.
5. Leave the Security setting "**Allow delivery of PDF and ZIP files**" as it is. Free accounts block PDF delivery on CDN URLs; the HMIS uses the API download endpoint, which is reported to serve them regardless. If PDFs still fail in testing (chapter 9), turning the setting on does **not** make HMIS documents public — they stay authenticated — but test again afterwards.

NOTE: What the Blueprint does not do

Neither `render.yaml` nor the HMIS creates or configures the Cloudinary account. The HMIS only uploads, links and deletes assets under `CLOUDINARY_FOLDER`.

4. Environment variables and secrets

The application reads the process environment only (`.env` files are not loaded). On Render, values come from the Blueprint, from Render itself, and from what you type into the dashboard. `backend/hmis/.env.example` and `frontend/.env.example` list every variable with placeholders.

4.1 Set by the Blueprint (group `hmis-settings: api, worker, beat`)

Variable	Value
<code>DJANGO_ENV</code>	<code>production</code>
<code>PYTHON_VERSION</code>	<code>3.12.8</code>
<code>TIME_ZONE</code>	<code>Africa/Lagos</code>
<code>DJANGO_MEDIA_STORAGE</code>	<code>cloudinary</code>
<code>CLOUDINARY_LINK_EXPIRY_SECONDS</code>	<code>900</code>
<code>HMIS_CLIENT_IP_HEADER</code>	<code>CF-Connecting-IP</code> (Render's proxy appends to X-Forwarded-For; Cloudflare in front of it overwrites this header)
<code>DJANGO_LOG_LEVEL</code>	<code>INFO</code>
<code>DATABASE_URL</code>	Wired from <code>hmis-db</code> (internal URL)
<code>REDIS_URL</code>	Wired from <code>hmis-redis</code> (internal URL)

4.2 Set by Render itself

RENDER, RENDER_SERVICE_TYPE, RENDER_EXTERNAL_HOSTNAME, RENDER_EXTERNAL_URL, PORT, WEB_CONCURRENCY. The onrender.com hostname and URL are added to ALLOWED_HOSTS and CSRF_TRUSTED_ORIGINS automatically; the Celery workers, which serve no HTTP, need no hostname.

4.3 You enter them (prompted when the Blueprint is applied)

Variable	Services	Value
DJANGO_SECRET_KEY	api, worker, beat — same value in all three	50+ random characters: <code>python3 -c "import secrets; print(secrets.token_urlsafe(64))"</code> . Render's own <i>generate</i> makes 44 characters, which the HMIS refuses.
CLOUDINARY_CLOUD_NAME, CLOUDINARY_API_KEY, CLOUDINARY_API_SECRET	api, worker, beat — same values in all three	From chapter 3. The secret only ever goes into Render's environment.
CORS_ALLOWED_ORIGINS	api	The static site's origin, e.g. <code>https://hmis-web.onrender.com</code> (https only)
HMIS_BASE_URL	api, worker	The React app's URL (links in admin emails)
VITE_API_BASE_URL	web (static)	The API's origin, e.g. <code>https://hmis-api.onrender.com</code> — public , compiled into the bundle; never a secret

The worker and Beat need the secret key and Cloudinary values because they load the same settings, which refuse to start without them.

4.4 Optional — add on a service's Environment page

Variable	Service(s)	Purpose
RESEND_API_KEY, HMIS_EMAIL_FROM, HMIS_ADMIN_EMAIL	api and worker	Admin email (sent by the worker; inline from the API if the queue is down)
HMIS_ADMIN_ALLOWED_NETWORKS	api	Django Admin only from these CIDR ranges (e.g. the hospital's public IP /32); others get 404
DJANGO_ALLOWED_HOSTS, DJANGO_CSRF_TRUSTED_ORIGINS	api	A custom API domain
DJANGO_SECURE_HSTS_SECONDS	api	Lower (e.g. 3600) for the first days; default one year
CLOUDINARY_FOLDER	api, worker, beat	Public-ID prefix (default <code>nmhs-hmis/media</code>); change only before the first upload
DB_CONN_MAX_AGE	api	Database connection reuse (default 60 s)

WARNING: Do not set on Render

DJANGO_DEBUG (true is refused) · DJANGO_LOG_DIR (files are lost; check `--deploy` warns W002) · DJANGO_MEDIA_STORAGE=filesystem (refused) · any secret in a VITE_* variable · S3/AWS variables (no longer used).

5. First deployment with the Render Blueprint

1. Push the repository to a **private** GitHub repository. Nothing in it holds a secret; `.env` files are gitignored.
2. Set up Cloudinary (chapter 3) and keep the cloud name, API key and API secret ready — enter them only into Render, never into a file or a chat.
3. In Render: **New → Blueprint**, connect GitHub, pick the repository and branch. Render reads `render.yaml` and lists six resources: `hmis-api`, `hmis-web`, `hmis-celery-worker`, `hmis-celery-beat`, `hmis-redis` and `hmis-db`. Review plans and prices on that screen.
4. Enter the prompted values (chapter 4.3). If the names `hmis-api` / `hmis-web` are free, the URLs will be `https://hmis-api.onrender.com` and `https://hmis-web.onrender.com`; if Render adds a suffix, correct `CORS_ALLOWED_ORIGINS`, `HMIS_BASE_URL` and `VITE_API_BASE_URL` afterwards and redeploy (the static site must be **rebuilt** to pick up `VITE_API_BASE_URL`).
5. Apply. Render creates **PostgreSQL** and **Key Value**, builds the **API** (install + `collectstatic`), runs `migrate` in the pre-deploy step — which also seeds departments (Procedure and Theatre as separate departments), stock locations, catalogues and price lists — starts Unicorn and waits for `/healthz/`. It builds the **static site** and starts the **Celery worker** and the single **Celery Beat**.
6. Open `https://hmis-api.onrender.com/healthz/` → `{"status": "healthy"}`; then `/admin/` shows a styled Django Admin login (WhiteNoise works).
7. In the API's **Shell** tab: `python manage.py check --deploy` — no errors (W005/W021 about HSTS subdomains/preload are expected while those stay off).
8. Check the worker and Beat logs show them connected (`celery@... ready, beat: Starting...`); from the API Shell, `celery -A hmis inspect ping` answers pong.

5.1 PostgreSQL and Key Value settings in the Blueprint

- `hmis-db`: PostgreSQL 16, database `hmis`, user `hmis`, `ipAllowList: []` (private network only). Set its storage size in the dashboard.
- `hmis-redis`: `maxmemoryPolicy: noeviction` so queued Celery tasks are never evicted; `ipAllowList: []`. It also holds the login-lockout counts, shared by every Unicorn worker. If it is down, the lockout fails open and email is sent inline — sign-in and clinical work continue.

6. The Super Admin account

1. In the API's Shell: `python manage.py createsuperuser`. Production password rules apply (10+ characters, not common, not numeric, not similar to the username).
2. Sign in to `https://<api>/admin/` → Users, open that account and set **Role = Super Admin**. Without a role an account reaches nothing in the HMIS.
3. Sign in to the React app with it and configure the hospital (User Manual chapter 24: hospital settings, departments and their staff — including Procedure staff — users, wards, beds, prices).
4. Keep few Super Admins; use Hospital Admin for day-to-day administration. Never put this password in an environment variable, a file or a chat.

7. Custom domains, CORS, CSRF and VITE_API_BASE_URL

1. API: Settings → Custom Domains → add e.g. `api.example-hospital.org`, create the DNS record. Render shows, wait for the certificate. Add the domain to `DJANGO_ALLOWED_HOSTS` and `https://api.example-hospital.org` to `DJANGO_CSRF_TRUSTED_ORIGINS` (Django Admin's sign-in form needs it).
2. Frontend: add e.g. `hmis.example-hospital.org` to the static site the same way. Add `https://hmis.example-hospital.org` to the API's `CORS_ALLOWED_ORIGINS` (comma-separated with the onrender origin) and update `HMIS_BASE_URL`.
3. Set the static site's `VITE_API_BASE_URL=https://api.example-hospital.org` and **rebuild** it.
4. Only once HTTPS has worked for some days, consider HSTS include-subdomains / preload — and only if **every** subdomain of the domain is HTTPS.

Document links point to `api.cloudinary.com`, never to the frontend's or the API's hostname, so custom domains do not affect them.

8. Celery worker and the single Celery Beat

- `hmis-celery-worker` runs the existing tasks: admin email (`apps.core.tasks.send_admin_email_task`, with retries and de-duplication) and the nightly stock tasks.
- `hmis-celery-beat` is the **only** scheduler: `numInstances: 1`, never autoscaled, never `-B` on the worker, never inside the web service. Two schedulers would run every nightly task twice. Its schedule file in `/tmp` only records last run times; losing it is harmless.
- Nightly tasks (every 24 h): `check_low_stock` and `check_expiring_batches`. As before, they compute lists only; the dashboards calculate stock alerts live.

9. Verifying documents upload and stay private

Do this with a **test patient** after the first deployment:

1. Upload a PDF and an image to the test patient's Medical Tests & Diagnostics record.
2. In the Cloudinary Console, confirm both assets are under `nmhs-hmis/media/medical_tests/...` as **raw** assets of type **authenticated**.
3. Open the document from the chart. The link opens from `https://api.cloudinary.com/v1_1/<cloud>/raw/download?...`
4. Take the asset's ordinary address (`https://res.cloudinary.com/<cloud>/raw/authenticated/...`) and open it without a signature: Cloudinary must **refuse** it.
5. Keep the download link and try it again after `CLOUDINARY_LINK_EXPIRY_SECONDS` (15 minutes by default): it must **fail**. Reopening the record gives a fresh link.
6. Sign in as a role that may not read the record (e.g. a cashier): the record — and so the link — must not be offered.
7. Search the API logs for the link's signature or the API secret: neither may appear.

IMPORTANT: Deleting records

As before, deleting or purging a record does not delete its file. With Cloudinary, the asset stays (private) in the account; remove it from the Cloudinary Console if the hospital's retention policy requires.

10. Health checks, logs and troubleshooting

10.1 Health check

GET /healthz/ answers {"status": "healthy"} (200) when the database answers, {"status": "unhealthy"} (503) when it does not — nothing else. It is unauthenticated, outside /api/, and exempt from the HTTPS redirect. Render's health check uses it; Cloudinary and Redis are deliberately not part of it.

10.2 Logs

- Every service's stdout/stderr is on its **Logs** tab. Each line carries a request ID (also returned in the X-Request-ID header and carried into Celery tasks); unexpected errors are logged with the RF- reference the user sees.
- **Never logged:** passwords (including what was typed as a username on a failed sign-in), tokens, Authorization headers, cookies, request bodies, query strings, the secret key, database/Redis/Cloudinary credentials, document links, and no patient records, diagnoses, results, prescriptions, notes, addresses or phone numbers.
- Retention is 7/14/30 days by workspace plan; add a **log stream** for longer.
- Render's own HTTP request logs (Pro workspaces and above) are written by Render's proxy and may include full URLs **with query strings** — i.e. patient searches. Limit who can view logs in the workspace.

10.3 Troubleshooting

Symptom	Likely cause	What to do
Start-up fails with ImproperlyConfigured	A required production variable is missing or unsafe	The log names the variable (never its value); set it and redeploy
Deploy cancelled in pre-deploy	migrate failed	Read the deploy log; the old release keeps serving. Fix and redeploy
Health check fails; /healthz/ unhealthy	API cannot reach PostgreSQL	Check DATABASE_URL is wired from hmis-db; same region
400 on the API's custom domain	Domain not in DJANGO_ALLOWED_HOSTS	Add it
CORS error in the browser	Origin missing from CORS_ALLOWED_ORIGINS, or http://	Add the exact https:// origin, no trailing slash
Requests go to the static site	VITE_API_BASE_URL wrong or not rebuilt	Fix it and rebuild the static site
CSRF error at Django Admin sign-in	Custom API domain not in DJANGO_CSRF_TRUSTED_ORIGINS	Add https://<api domain>
Django Admin 404	HMIS_ADMIN_ALLOWED_NETWORKS excludes you	Add your network or unset it
check --deploy: hmis.E011	HMIS_CLIENT_IP_HEADER not set	Set CF-Connecting-IP
check --deploy: hmis.E012	Documents not on Cloudinary	Set DJANGO_MEDIA_STORAGE=cloudinary and the three CLOUDINARY_* values
Document upload fails	Wrong Cloudinary credentials, or file over the plan's raw size limit	Check the CLOUDINARY_* values (same on api/worker/beat); check the plan
Document link fails	Expired (default 15 min)	Reopen the record for a fresh link
PDF documents will not open	Cloudinary PDF delivery restriction	See chapter 3.3, step 5

Symptom	Likely cause	What to do
Admin email not sent	RESEND_* missing on api or worker	Set them on both
Celery idle	Worker/Beat stopped or REDIS_URL not wired	Check both logs; <code>celery -A hmis inspect ping</code>
User sees "... RF-xxxxxxx"	An unexpected error	Search the API logs for the reference

11. Backups and recovery

11.1 Database

- Render PostgreSQL (paid) has **point-in-time recovery**: past 3 days (Hobby workspace) or 7 days (Pro+). Recovery creates a **new** database at the chosen time; check it, then point DATABASE_URL at it.
- **Logical exports** from the dashboard (paid) are kept 7 days — download them and store them encrypted outside Render for longer retention.
- Own daily dumps: temporarily allow your trusted IP /32 on the database, run `pg_dump -Fc "<external database URL>" > hmis-$(date +%F).dump`, encrypt and store it outside Render, then remove the IP again.
- **Restore test monthly** into a scratch database; compare counts of patients, charges and payments.

11.2 Documents

Documents live in Cloudinary, not in the database backups. Use the backup / restore options your Cloudinary plan offers. The database stores each document's name, which is its public ID under CLOUDINARY_FOLDER; because the HMIS never deletes Cloudinary assets when records are purged, a restored database still finds its documents.

11.3 Disaster recovery

Re-apply the Blueprint (new workspace or region if needed), restore the database from point-in-time recovery or a dump, point the Cloudinary variables at the same product environment, re-enter the secrets, re-point DNS. Rehearse it at least once a year.

12. Updates, security and patient-data protection

12.1 Updating the HMIS

1. Take a logical export (or note the point-in-time-recovery timestamp).
2. Merge to the deploy branch. Render rebuilds, runs `migrate` in pre-deploy, and keeps the old release serving if anything fails.
3. Watch the API logs for RF- errors; open `/healthz/`.
4. Roll back with **Manual Deploy → a previous commit**. If a migration already ran, restore the database to before the deploy first.

12.2 Security checklist

- All resources on paid plans appropriate for patient data.
- Database and Key Value private (ipAllowList: []).
- Cloudinary: dedicated product environment, 2FA for every console user, no unsigned upload presets, API secret only in Render, short CLOUDINARY_LINK_EXPIRY_SECONDS.
- DJANGO_SECRET_KEY unique, 50+ characters, identical on api/worker/beat.
- CORS_ALLOWED_ORIGINS lists only the HMIS frontend origin(s).
- HMIS_CLIENT_IP_HEADER=CF-Connecting-IP; check --deploy passes.
- Few Super Admins with strong passwords; optional HMIS_ADMIN_ALLOWED_NETWORKS.
- Render workspace members limited, 2FA for each; GitHub repository private.
- The repository's tracked db.sqlite3.bak-20260903-073530 and dump.rdb reviewed for patient data and removed from version control.
- Point-in-time-recovery window known; first export downloaded; a restore tested.

12.3 Patient-data protection

- Patient data lives in two places: Render PostgreSQL (records) and Cloudinary (documents). Both providers' data-protection terms and storage locations are the hospital's decision.
- Documents are never public: authenticated assets, expiring links, existing permissions.
- Logs hold identifiers only, never clinical content or credentials.
- The audit trail (AuditLog, Django Admin → Core → Audit logs) records sign-in, sign-out and every clinical and financial action.

13. Smoke-test the HMIS after deploying

With **test patients only**, walk each critical workflow once:

1. Reception: register a patient, print the patient card, bill a consultation, take a payment and print the receipt.
2. Nurse: accept the vitals route, record vitals and a nursing note, send to doctor.
3. Doctor: open the chart, write a note, refer for a laboratory test and a **Procedure**, prescribe a drug.
4. Laboratory: accept, enter a result, submit, verify; the doctor sees it on the chart.
5. Procedure staff (a doctor/nurse on the **Procedure** department): accept, record with materials, Save & mark done; print the Procedure Record.
6. Documents: the privacy checks in chapter 9.
7. Pharmacy: pay the prescription line, dispense; a POS sale with receipt.
8. Cashier: a refund, a service cancellation, the financial report.
9. Admissions: admit to a bed, discharge, print the discharge letter.
- 10 Sign out; five wrong passwords lock the username for 5 minutes from that client.
- .

Then delete the test patients (Super Admin, Django Admin purge) before go-live, and remove their test documents from the Cloudinary Console.

14. What has and has not been tested

Tested locally (before the switch to Cloudinary)	Not tested
Backend and frontend suites; check, check --deploy, makemigrations --check; frontend build	Any Render deployment
Production settings with Render's variables in a clean process; collectstatic (162 files, manifest)	The Cloudinary storage — its tests in <code>apps/core/tests/test_production_settings.py</code> are written but not run
Gunicorn behind a simulated proxy: WhiteNoise CSS 200, /healthz/ 503 without a database, HTTP→HTTPS 301 without a loop, unknown host 400, CORS for the static site only, security headers present	A real Cloudinary account, PostgreSQL server, Key Value, Render's health checker

Before relying on the Cloudinary storage, run its tests after `./venv/bin/pip install -r requirements.txt` installs the `cloudinary` package, then do chapter 9 on Render.

Appendices

A. Local development (unchanged)

```
cd backend/hmis && ./venv/bin/python manage.py runserver      # SQLite, db.sqlite3
cd frontend && npm run dev                                     # Vite :5173, proxies /api and /media
```

Nothing has to be installed or set: no PostgreSQL, Redis, Cloudinary or Render. `VITE_API_BASE_URL` stays unset, so the app calls `/api` through the Vite proxy, and uploads stay in `backend/hmis/media/`.

B. Useful commands (API Shell on Render, or locally)

```
python manage.py check --deploy
python manage.py showmigrations
python manage.py createsuperuser
python manage.py billing_integrity [--strict]
python manage.py seed_lab_catalogue          # re-runnable; never overwrites edits
celery -A hmis inspect ping
```

C. Files that define the deployment

File	Role
<code>render.yaml</code>	Blueprint: services, commands, wired variables, prompts
<code>backend/hmis/hmis/environment.py</code>	Environment rules: database, hosts, CORS, client IP, media storage, admin networks
<code>backend/hmis/hmis/settings.py</code>	Settings built from those rules; logging; WhiteNoise
<code>backend/hmis/apps/core/storage.py</code>	Private Cloudinary storage (authenticated raw, expiring links)
<code>backend/hmis/apps/core/checks.py</code>	<code>check --deploy</code> errors <code>hmis.E001-E012</code>
<code>backend/hmis/.env.example</code> , <code>frontend/.env.example</code>	Every variable, placeholders only
<code>docs/DEPLOYMENT.md</code>	The same guide in Markdown
<code>docs/pdf-source/</code>	Source of this PDF

D. Departments seeded by migrations

Reception, Consultation, Laboratory, Pharmacy, Radiology / Ultrasound, Eye, **Theatre**, **Procedure**, Clinicals (Nursing) and Maternity. **Procedure and Theatre are separate departments**, each with its own staff list and revenue column; membership of one grants nothing in the other.

E. Self-hosted alternative (not the primary method)

The same code still runs on a self-managed server: `DJANGO_MEDIA_STORAGE=filesystem` with `/media/` served by a reverse proxy restricted to the hospital network, `HMIS_CLIENT_IP_HEADER` unset and the proxy overwriting `X-Forwarded-For`. That set-up is no longer documented step by step.

F. Support information

Record for your installation: the hospital's HMIS owner, the technical administrator on call, the Render workspace owner, the Cloudinary account owner, the domain registrar, and where recovery credentials are kept. Keep this in the hospital's secure documentation, not in the repository.